

Experiment - 5

Name: K. SRI LASYA

Regno: 192472216

Course Code: MLA0305

Slot: B

Title

Dynamic Programming using Policy Iteration and Value Iteration

AIM

To implement **Policy Iteration and Value Iteration** using Dynamic Programming and determine the optimal policy for a Reinforcement Learning environment.

PROCEDURE

1. Import NumPy, Matplotlib, and Gymnasium.
 2. Create the FrozenLake environment.
 3. Define the state and action spaces.
 4. Initialize the value function and policy.
 5. Implement the Value Iteration algorithm.
 6. Apply Bellman updates until the value function converges.
 7. Implement Policy Iteration using policy evaluation and policy improvement.
 8. Obtain the optimal policies from both methods.
 9. Compare the policies obtained using the two algorithms.
 10. Visualize the optimal state values.
-

4. PYTHON SOURCE CODE

```
import gymnasium as gym
import numpy as np
import matplotlib.pyplot as plt

env = gym.make("FrozenLake-v1", is_slippery=False)

P = env.unwrapped.P

num_states = env.observation_space.n
```

```
num_actions = env.action_space.n

gamma = 0.99
theta = 1e-8

# -----
# VALUE ITERATION
# -----

V_value = np.zeros(num_states)

while True:

    delta = 0

    for state in range(num_states):

        action_values = []

        for action in range(num_actions):

            value = 0

            for probability, next_state, reward, done in P[state][action]:

                value += probability * (
                    reward + gamma * V_value[next_state]
                )

            action_values.append(value)

        best_value = max(action_values)
```

```
delta = max(  
    delta,  
    abs(best_value - V_value[state])  
)  
  
V_value[state] = best_value  
  
if delta < theta:  
    break  
  
policy_value = np.zeros(num_states, dtype=int)  
  
for state in range(num_states):  
  
    action_values = []  
  
    for action in range(num_actions):  
  
        value = 0  
  
        for probability, next_state, reward, done in P[state][action]:  
  
            value += probability * (  
                reward + gamma * V_value[next_state]  
            )  
  
        action_values.append(value)  
  
    policy_value[state] = np.argmax(action_values)
```

```
# -----  
# POLICY ITERATION  
# -----  
  
policy = np.zeros(num_states, dtype=int)  
V_policy = np.zeros(num_states)  
  
while True:  
  
    # Policy Evaluation  
  
    while True:  
  
        delta = 0  
  
        for state in range(num_states):  
  
            old_value = V_policy[state]  
  
            action = policy[state]  
  
            value = 0  
  
            for probability, next_state, reward, done in P[state][action]:  
  
                value += probability * (  
                    reward + gamma * V_policy[next_state]  
                )  
  
            V_policy[state] = value  
  
        delta = max(  

```

```
        delta,
        abs(old_value - value)
    )

    if delta < theta:
        break

# Policy Improvement

stable = True

for state in range(num_states):

    old_action = policy[state]

    action_values = []

    for action in range(num_actions):

        value = 0

        for probability, next_state, reward, done in P[state][action]:

            value += probability * (
                reward + gamma * V_policy[next_state]
            )

        action_values.append(value)

    policy[state] = np.argmax(action_values)

    if old_action != policy[state]:
```

```
        stable = False

    if stable:
        break

# -----
# OUTPUT
# -----

print("Value Iteration Policy:")
print(policy_value)

print("\nPolicy Iteration Policy:")
print(policy)

print("\nValue Function from Value Iteration:")

for i in range(num_states):
    print(f"State {i}: {V_value[i]:.3f}")

# -----
# VISUALIZATION
# -----

states = [f"S{i}" for i in range(num_states)]

plt.figure(figsize=(10, 6))

bars = plt.bar(
    states,
```

```
V_value,  
color="cornflowerblue",  
edgecolor="black",  
linewidth=1.2  
)  
  
plt.title(  
    "Dynamic Programming - Value Function",  
    fontsize=15,  
    fontweight="bold"  
)  
  
plt.xlabel("States", fontsize=12)  
plt.ylabel("State Value", fontsize=12)  
  
plt.grid(  
    axis="y",  
    linestyle="--",  
    alpha=0.5  
)  
  
for bar in bars:  
  
    height = bar.get_height()  
  
    plt.text(  
        bar.get_x() + bar.get_width() / 2,  
        height + 0.02,  
        f"{height:.2f}",  
        ha="center",  
        fontsize=9,  
        fontweight="bold"
```

)

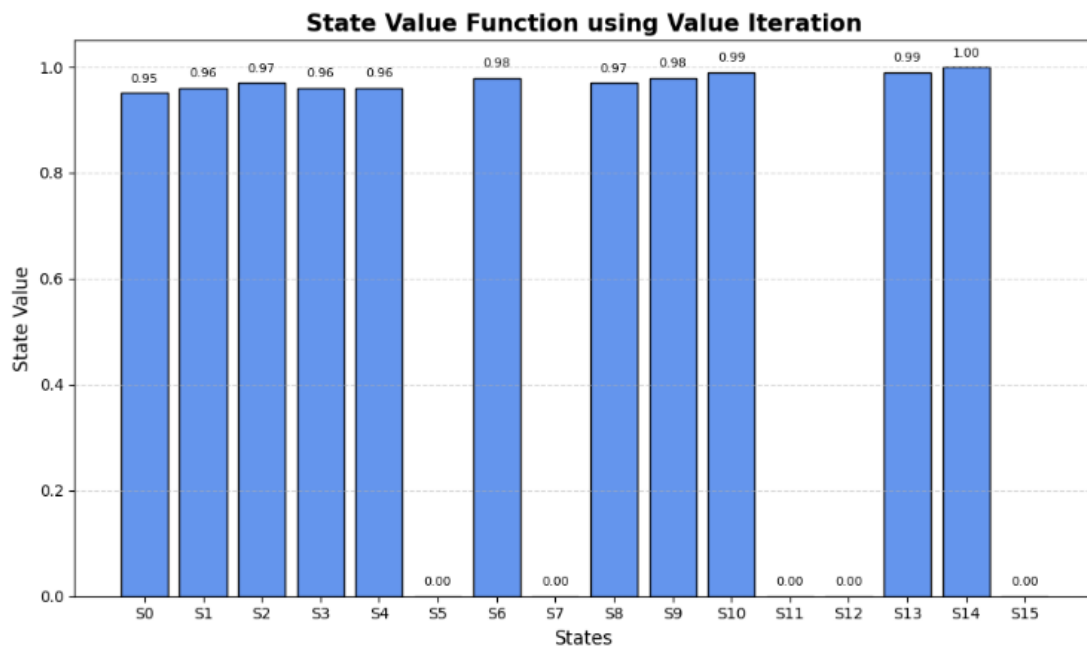
```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:

..

```
State Values
State 0: 0.951
State 1: 0.961
State 2: 0.970
State 3: 0.961
State 4: 0.961
State 5: 0.000
State 6: 0.980
State 7: 0.000
State 8: 0.970
State 9: 0.980
State 10: 0.990
State 11: 0.000
State 12: 0.000
State 13: 0.990
State 14: 1.000
State 15: 0.000
```



5. VISUALIZATION

The bar graph represents the **optimal state-value function obtained using Value Iteration.**

- **X-axis:** States S0–S15
- **Y-axis:** State Value
- **Color:** Cornflower Blue
- The graph shows the estimated value of each state under the optimal policy.

6. SUMMARY TABLE

Component	Description
Environment	FrozenLake-v1
Methods	Policy Iteration and Value Iteration
Discount Factor	0.99
Convergence Threshold	1e-8
State Space	16 states
Action Space	4 actions
Visualization	State-value bar graph

7. RESULT

Policy Iteration and Value Iteration were successfully implemented using Dynamic Programming. Both algorithms determine an optimal policy for the FrozenLake environment, and the resulting state values were visualized using a bar graph.